Week 6: AI Python Lab Manual: Informed Search

Lab Objective

By the end of this lab, students will be able to:

- Understand informed search strategies in Artificial Intelligence.
- Understand the role of heuristic functions in guiding search.
- Design a heuristic function for a real-world route-planning problem.
- Implement Greedy Best-First Search (GBFS).
- Implement A* Search.
- Understand the difference between `g(n)`, `h(n)`, and `f(n)`.
- Compare GBFS and A* on the same real-world problem.
- Identify practical scenarios where informed search algorithms can be used.

Prerequisites

- Python programming basics.
- Lists, dictionaries, sets, and priority queues.
- Functions and loops.
- Basic understanding of graphs.
- Understanding of nodes and edges.
- Basic understanding of cost functions.

Introduction to Informed Search

Scenario: Food Delivery Route Planning

Consider a food delivery application.

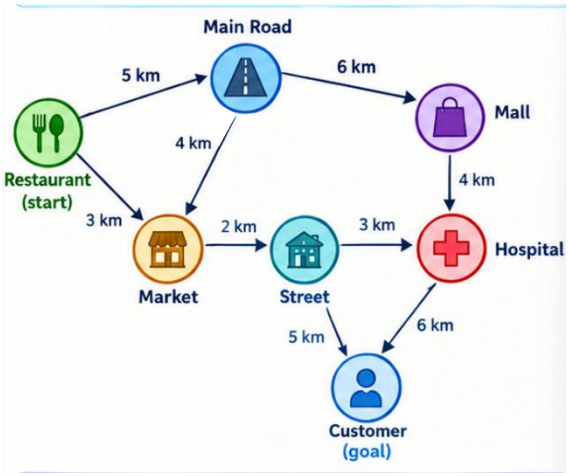
A rider has picked up an order from a restaurant and needs to deliver it to a customer. There are several roads that the rider can take.

The application represents the road network as a **graph**.

In this graph:

- A **node** represents a location.
- An **edge** represents a road connecting two locations.
- The **edge cost** represents the travel distance of that road.
- The **start node** represents the restaurant.
- The **goal node** represents the customer's location.

For example:



The rider needs to find a path from:

`Restaurant` → `Customer`

There may be multiple possible routes.

The challenge is:

> ***How can an AI system decide which location to explore next without blindly exploring every possible route?***

This is where **informed search** becomes useful.

What is Informed Search?

- Informed searches use additional problem-specific information to guide the search toward the goal.
- This additional information is represented by a **heuristic function**.
- The heuristic function is commonly written as: **`h(n)`**

where `h(n)` **estimates** the cost of reaching the goal from the current node **`n`**.

For a delivery route, a natural heuristic is:

`Estimated straight-line distance from the current location to the customer`

The heuristic does not necessarily represent the actual road distance.

Instead, it **provides an estimate that helps the search algorithm** determine which locations appear promising.

To understand the above Example in detail:

Term	Meaning	Delivery Example
Node	A state/location in the search space	Restaurant, Market, Mall
Edge	Connection between	Road

	two nodes	
Edge Cost	Cost of travelling along an edge	Road distance in km
Start Node	Initial state	Restaurant
Goal Node	Desired state	Customer
`g(n)`	Actual cost from start to node `n(current node)`	Distance already travelled
`h(n)`	Estimated cost from node `n` to goal	Estimated remaining distance
`f(n)`	Estimated total cost	`g(n) + h(n)`

Important Difference

GBFS uses: `h(n)`

while A* uses: `f(n) = g(n) + h(n)`

Therefore:

GBFS → Considers how close a location appears to the goal.

A* → Considers both the cost already travelled and the estimated cost remaining.

Designing a Heuristic Function

The heuristic depends on the **structure of the problem** and what **"distance/cost"** means in that problem.

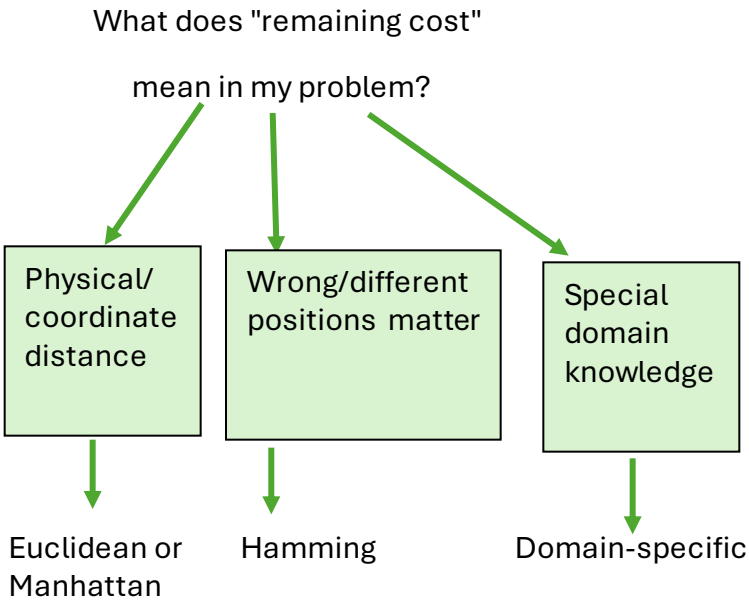
The easiest decision Table:

Problem structure	**Suitable heuristic**
Open 2D space / geographic coordinates	**Euclidean**
Grid where movement is only up/down/left/right	**Manhattan**
Goal is fastest arrival	**Estimated travel time**
Problem has special domain information	**Domain-specific heuristic**

*A **heuristic** is an estimate that gives the search algorithm **useful guidance** toward the goal.*

So how should you choose?

Consider simple decision process:



"Choose a heuristic that represents the remaining cost of your particular problem."

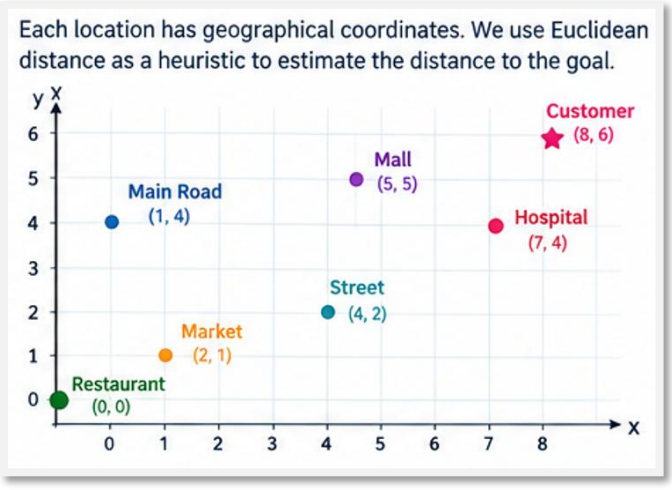
For example, **Manhattan** makes sense in a grid because the allowed movement is horizontal/vertical. **Euclidean** makes sense when straight-line spatial separation is meaningful. **Hamming** makes sense when the difference between states is naturally measured by the number of differing positions.

And for **A*** there is one additional requirement: if you want the usual optimality guarantee, the heuristic needs to satisfy the relevant **admissibility** assumptions i.e. it should not overestimate the actual remaining cost.

Scenario: The delivery application knows the approximate geographical coordinates of each location.

For example:

```
Restaurant → (0, 0)
Market     → (2, 1)
Main Road  → (1, 4)
Mall       → (5, 5)
Street     → (4, 2)
Hospital   → (7, 4)
Customer   → (8, 6)
```



We can use these coordinates to estimate the distance between a location and the customer.

A simple heuristic for this problem is **Euclidean distance**:

`h(n) = straight-line distance from n to the goal`

The formula is:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

where:

- (x_1, y_1) = coordinates of the current location.
- (x_2, y_2) = coordinates of the goal.

Why Do We Pass the Goal to the Heuristic?

The heuristic is supposed to calculate:

```
distance(current location → goal)
```

Therefore, it needs to know **both** locations.

For example:

```
heuristic("Mall", "Customer")
```

means:

> Estimate the distance from Mall to Customer.

If the goal changes, the heuristic can calculate a different value.

For example:

```
heuristic("Mall", "Customer")
heuristic("Mall", "Hospital")
```

would calculate two different distances.

Designing the Heuristic

For a road navigation problem, possible heuristics include:

- Straight-line/**Euclidean** distance to the destination.
- **Manhattan** distance in a city-grid environment.
- Estimated travel time to the destination.
- A customized domain-specific estimate.

The heuristic should provide a reasonable estimate of the remaining cost. Ideally, for A*, an admissible heuristic should **not overestimate** the actual remaining cost.

Python Implementation of the Heuristic

```
import math

# Coordinates of each location
locations = {
    "Restaurant": (0, 0),
    "Market": (2, 1),
    "Main_Road": (1, 4),
    "Mall": (5, 5),
    "Street": (4, 2),
    "Hospital": (7, 4),
    "Customer": (8, 6)
}


def heuristic(current, goal):

    x1, y1 = locations[current]
    x2, y2 = locations[goal]

    distance = math.sqrt(
        (x2 - x1) ** 2 +
        (y2 - y1) ** 2
    )

    return distance
```

Test the Heuristic

```
print(
    "Estimated distance from Mall to Customer:",
    round(heuristic("Mall", "Customer"), 2),
    "km")
```

Expected Output

```
Estimated distance from Mall to Customer: 3.16 km
```

The value `3.16 km` is the **estimated straight-line distance**, not necessarily the actual road distance.

Why Use Straight-Line Distance?

Suppose a road network forces the rider to travel around buildings, roads, or other obstacles.

The actual road distance may be: **`4 km`**

while the straight-line distance is: **`3.16 km`**

The straight-line distance is therefore a **lower-bound estimate** of the actual road distance in a normal road-network setting.

This makes it a useful heuristic for A*.

Important Note

- A heuristic is an **estimate**. It does not have to be the exact remaining cost.
- A good heuristic should provide useful guidance while remains appropriate for the search problem.

Greedy Best-First Search (GBFS)

Consider the following scenario:

Imagine the delivery rider follows this strategy:

> "At every step, choose the location that appears closest to the customer."

This is the basic idea of **Greedy Best-First Search**.

GBFS selects the node with the **smallest heuristic value**:

```
Priority = h(n)
```

It does **not** consider how much distance the rider has already travelled.

Therefore:

```
GBFS → uses h(n)
```

Example

Suppose the rider has reached two possible locations:

Location	Estimated distance to Customer
Market	7.81 km
Main Road	7.28 km

GBFS chooses: `Main Road`

because: `7.28 < 7.81`

GBFS checks the available neighboring nodes, but when choosing among them, it considers only their estimated distance to the goal $h(n)$, not the cost already spent reaching them $g(n)$.

GBFS Python Implementation

Algorithm

1. Put the starting node into a priority queue.
2. Assign priority using $h(n)$.
3. Remove the node with the smallest heuristic value.
4. If it is the goal, stop.
5. Otherwise, examine its unvisited neighbors.
6. Add the neighbors to the priority queue using their heuristic values.
7. Keep track of each node's parent so that the final path can be reconstructed.
8. Repeat until the goal is reached or no nodes remain.

Pseudocode

```
GBFS(start, goal):

    create an empty priority queue
    insert start with priority h(start)

    mark start as having no parent

    while priority queue is not empty:

        remove node with smallest h(n)

        if node is goal:
            reconstruct and return path

        if node has already been explored:
            continue

        mark node as explored

        for each neighbor of node:

            if neighbor has not been explored:

                store node as neighbor's parent

                insert neighbor into priority queue
                using h(neighbor)
```

return failure

Helper Function

```
def reconstruct_path(came_from, current):

    path = []

    while current is not None:

        path.append(current)
        current = came_from[current]

    path.reverse()

    return path
```

A* Search

Now suppose the delivery application wants to select a route based on both:

1. The distance already travelled.
2. The estimated remaining distance.

A* Search uses: `f(n) = g(n) + h(n)`

where:

`g(n) = actual cost from the restaurant to node n`

`h(n) = estimated cost from node n to the customer`

`f(n) = estimated total route cost through n`

Example

Suppose two possible locations are available:

Location	g(n) Actual Cost So Far	h(n) Estimated Remaining	f(n)
Main Road	4.12 km	7.28 km	11.40 km
Market	2.24 km	7.81 km	10.05 km

GBFS compares only:

```
h(Main Road) = 7.28
h(Market) = 7.81
```

and therefore chooses: `Main Road`

However, A* compares:

```
f(Main Road) = 4.12 + 7.28 = 11.40
f(Market)    = 2.24 + 7.81 = 10.05
```

and therefore chooses: `Market`

This is the **fundamental** difference between the algorithms.

GBFS:
"Which location looks closest?"

A*:
"Which location appears to have the lowest total cost from start to goal?"

Python Implementation of A*

Algorithm

1. Put the starting node into a priority queue.
2. Set $g(start) = 0$.
3. Calculate its priority using $f(start) = g(start) + h(start)$.
4. Remove the node with the smallest $f(n)$.
5. If it is the goal, reconstruct the path.
6. Examine its neighbors.
7. Calculate the new cost of reaching each neighbor.
8. If the new route is cheaper than the previously known route, update:
    - the neighbor's g-cost
    - its parent
    - its priority $f(n)$
9. Repeat until the goal is reached or the frontier becomes empty.

Pseudocode

```
A*(start, goal):
    create an empty priority queue
```

```
g(start) = 0
insert start with priority g(start) + h(start)
mark start as having no parent
while priority queue is not empty:
   remove node with smallest f(n)
   if node is goal:
      reconstruct and return path
   for each neighbor of node:
      calculate tentative_g
      if neighbor has never been reached
      OR tentative_g is cheaper:
         update g(neighbor)
         calculate:
            f(neighbor) = g(neighbor) + h(neighbor)
         update neighbor's parent
         insert neighbor into priority queue
return failure
```

The graph contains two major possible routes:

Route 1: Restaurant → Main Road → Mall → Customer

and:

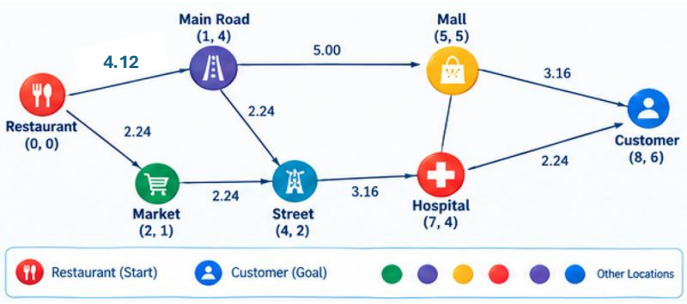
Route 2: Restaurant → Market → Street → Hospital → Customer

Their approximate costs are:

```
Route 1 = 4.12 + 5.00 + 3.16
        = 12.28 km

Route 2 = 2.24 + 2.24 + 3.16 + 2.24
        = 9.88 km
```

Therefore, **Route 2 is the shorter route**.




```
        path.append(current)
```

```
        current = came_from.get(current)

    path.reverse()

    return path
```

Understanding the Result

This example demonstrates an important property of informed search.

GBFS chooses:

`Restaurant → Main Road → Mall → Customer`

Why?

Because at each decision point, it prefers the location with the **smallest estimated distance to the customer**.

However, this route has a total cost of approximately:

`12.28 km`

A* chooses:

`Restaurant → Market → Street → Hospital → Customer`

with a total cost of approximately:

`9.88 km`

Why?

Because A* considers both:

`g(n) = cost already travelled`

and:

`h(n) = estimated remaining cost`

Therefore:

`f(n) = g(n) + h(n)`

GBFS vs A*: Conceptual Comparison

Feature	GBFS	A*
Uses heuristic `h(n)`	Yes	Yes
Uses actual cost `g(n)`	No	Yes
Priority value	`h(n)`	`g(n) + h(n)`
Considers cost already travelled	No	Yes
Guaranteed optimal with this setup	No	Yes, when the heuristic is admissible and the usual A* assumptions hold
Main idea	Choose what looks closest	Balance cost already incurred and estimated remaining cost
Delivery interpretation	"Which location looks closest to the customer?"	"Which route currently has the lowest estimated total cost?"

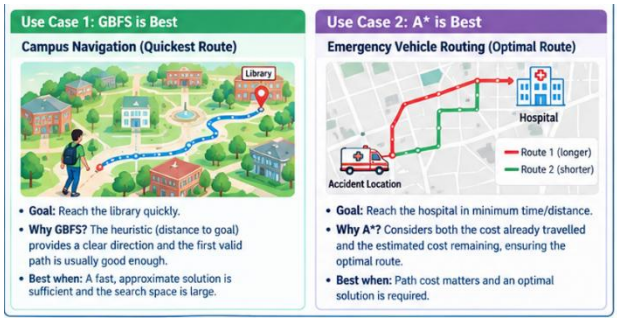
Important Practical Distinction

You should not conclude that: `A* is always better than GBFS.` This is too broad. The algorithms serve different purposes.

GBFS

GBFS can be useful when:

- A quick solution is preferred.
- An approximate route is acceptable.
- The search space is large.
- The heuristic provides useful guidance.



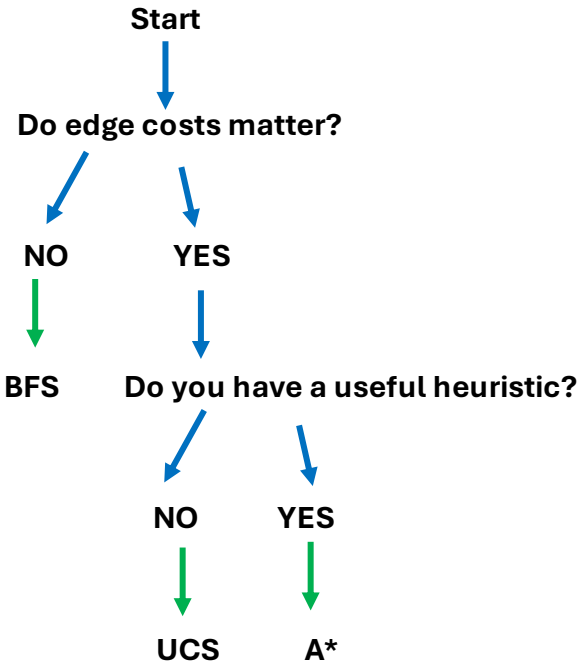
A*

A* is appropriate when:

- Path cost matters.
- We want to find an optimal path under the algorithm's assumptions.
- A suitable heuristic is available.
- Both previous cost and estimated remaining cost should influence the decision.

How do we decide which algorithm we should use in our system?

DECISION STRATEGY:



Where does GBFS fit?

GBFS is useful when: Getting toward the goal quickly is more important than guaranteeing the cheapest path.

Example code links:

https://www.geeksforgeeks.org/artificial-intelligence/greedy-best-first-search-in-ai/

https://www.analyticsvidhya.com/blog/2024/06/understanding-the-greedy-best-first-search-gbfs-algorithm-in-python/

https://onecompiler.com/python/3xdaztfta

https://www.mygreatlearning.com/blog/a-search-algorithm-in-artificial-intelligence/